

Apresentação: Fase 1 - Sistema de Pagamentos e Logística

Data: 12 de Maio de 2026 **Projeto:** Car Parts Marketplace (Japão) **Versão:** 1.0

1. Resumo Executivo

Este documento apresenta a primeira fase do sistema de pagamentos integrado com logística para o marketplace de peças automotivas. O sistema já possui a estrutura base implementada e está pronto para as próximas etapas de desenvolvimento.

Objetivos da Apresentação

1. Apresentar o estado atual do sistema de pagamentos
 2. Detalhar o fluxo do usuário do checkout
 3. Discutir modelos de implementação de logística
 4. Definir próximos passos
-

2. Estado Atual do Sistema de Pagamentos

2.1 O que já está implementado

Frontend

- **Checkout Page** (`src/pages/PaymentCheckout.tsx`)
 - Formulário de informações de entrega
 - Seleção de método de pagamento (Cartão/PIX)
 - Resumo do pedido com breakdown de taxas
 - Integração com fluxo de compra

- **Purchase Flow Component** (`src/components/PurchaseFlow.tsx`)

- Fluxo simplificado de compra
- Exibição de taxas e comissões
- Confirmação de pagamento

Backend (Edge Functions)

- **stripe-checkout** (`supabase/functions/stripe-checkout/index.ts`)

- Criação de sessão de checkout Stripe
- Criação de contas conectadas (Stripe Connect)
- Portal de payouts para vendedores
- Cálculo de taxas automático

- **stripe-webhook** (`supabase/functions/stripe-webhook/index.ts`)

- Processamento de webhooks de pagamento
- Confirmação automática de transações

- **transactions** (`supabase/functions/transactions/index.ts`)

- CRUD completo de transações
- Listagem filtrada por papel (buyer/seller/admin)
- Atualização de status de pagamento e fulfillment

Estrutura de Taxas (Implementada)

Componente	Taxa
Comissão Plataforma	10%
Taxa Stripe	2.9% + ¥30 (fixo)
Valor Vendedor	Total - (Comissão + Taxa Stripe)

2.2 Estrutura de Dados

Tabela: transactions

```
{  
  id: string (UUID)
```

```

part_id: string (FK)
buyer_id: string (FK - profiles)
seller_id: string (FK - profiles)
amount: number
commission_rate: number
commission_amount: number
platform_fee: number
seller_net: number
payment_status: 'pending' | 'paid' | 'failed' | 'refunded'
fulfillment_status: 'pending' | 'shipped' | 'delivered' | 'completed'
paid_at: timestamp
shipped_at: timestamp
delivered_at: timestamp
tracking_code: string
stripe_payment_id: string
created_at: timestamp
updated_at: timestamp
}

```

3. Fluxo do Usuário - Checkout

3.1 Fluxo Completo



3.2 Detalhamento das Etapas

Etapa 1: Seleção do Produto

- Usuário acessa catálogo

- Seleciona peça desejada
- Visualiza detalhes do produto
- Clica em "Comprar" ou inicia negociação

Etapa 2: Checkout - Informações de Entrega

Campos obrigatórios: - Nome completo - E-mail - Telefone - Endereço completo - Cidade - Estado/Região - CEP/Postal Code

Funcionalidades: - Validação de dados em tempo real - Pré-preenchimento para usuários logados - Suporte a endereços internacionais

Etapa 3: Forma de Pagamento

Opções disponíveis: - **Cartão de Crédito** - Pagamento parcelado ou à vista via Stripe - **PIX** - Pagamento instantâneo (futuro)

Resumo do pedido:

```
Peça: [Nome do produto]
Preço: ¥ XX,XXX
Taxa plataforma (10%): -¥ X,XXX
Taxa pagamento: -¥ XXX
Total: ¥ XX,XXX
```

Etapa 4: Processamento

- Criação da transação no banco
- Redirecionamento para checkout Stripe
- Processamento do pagamento
- Atualização de status

Etapa 5: Confirmação

- Exibição de confirmação de sucesso
- Notificação ao vendedor
- Instruções para próximos passos
- Link para acompanhamento do pedido

Etapa 6: Pós-Compra (Dashboard)

- Lista de compras/vendas do usuário
- Status em tempo real de cada transação
- Informações de rastreamento
- Histórico completo

3.3 Fluxo Técnico (API)

```
// 1. Criar transação
POST /functions/v1/transactions/create
{ part_id, amount }
// Response: { transaction_id, fees }

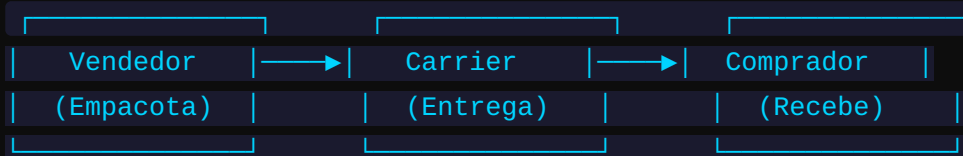
// 2. Criar sessão Stripe
POST /functions/v1/stripe-checkout/create-checkout
{ transaction_id, part_id, buyer_id, seller_id, amount }
// Response: { url: "https://checkout.stripe.com/..." }

// 3. Webhook - Confirmação pagamento
POST /functions/v1/stripe-webhook
{ type: "checkout.session.completed", data: { transaction_id } }
// Atualiza: payment_status = 'paid', paid_at = NOW
// Atualiza: part.status = 'sold'
// Notifica: vendedor
```

4. Sistema de Logística - Discussão

4.1 Modelos Possíveis

Opção A: Vendedor Envia Diretamente (Direct Ship)

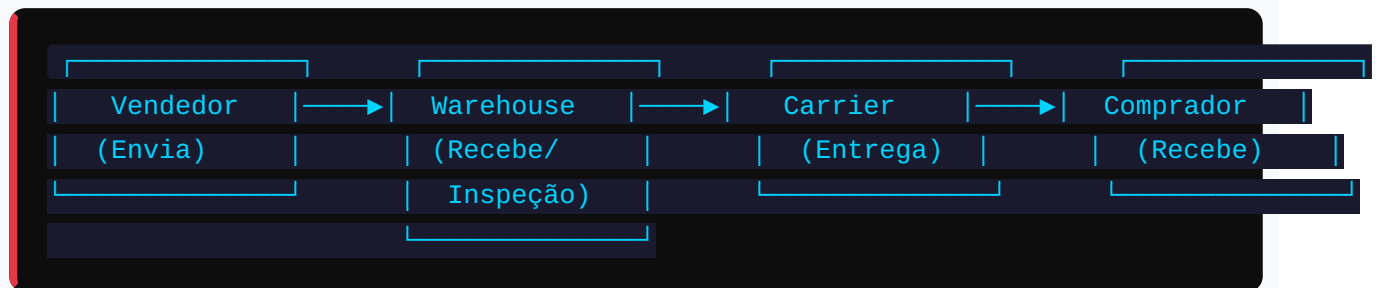


Prós: - Modelo mais simples de implementar - Menor custo operacional - Flexibilidade para vendedores

Contras: - Qualidade de packaging variável - Tracking inconsistente - Dificuldade de padronização

Necessário: - Formulário de endereço completo no checkout - Sistema de geração de código de rastreamento - Interface para vendedor registrar envio

Opção B: Warehouse/Fulfillment Center



Prós: - Qualidade padronizada - Tracking unificado - Possibilidade de combine shipping

Contras: - Custo operacional alto - Necessidade de parceiros/logística - Tempo adicional de processamento

Necessário: - Parceria com warehouse no Japão - Sistema de gestão de estoque - Contratos com carriers

4.2 Integração com Carriers Japoneses

Principais carriers do Japão:

Carrier	Nome Japonês	Características
Yamato Transport	ヤマト運輸	Maior rede, TA-Q-BIN
Sagawa Express	佐川急便	Entrega rápida, SCP
Japan Post	日本郵便	Cobertura nacional, econômico

Opções de implementação:

1. Manual (Fase 1)

- Vendedor registra código de rastreamento manualmente
- Sistema exibe código no dashboard do comprador
- Link direto para tracking do carrier

2. API Integration (Fase 2)

- Integração direta com APIs dos carriers
- Geração automática de labels
- Tracking automático de status
- Notificações push

3. Marketplace Aggregator (Fase 3)

- Contratar serviço de agregação
- Múltiplos carriers em uma plataforma
- Gerenciamento centralizado

4.3 Campos Necessários para Logística

Informações do Comprador (Checkout): - Nome completo (obrigatório) - Telefone (obrigatório - para entrega) - Endereço completo (obrigatório) - Cidade (obrigatório) - Prefecture/Estado (obrigatório) - Postal Code (obrigatório)

Informações do Vendedor (Shipping): - Carrier utilizado - Código de rastreamento - Data de envio - Número de volumes (para combined orders) - Estimativa de entrega

5. Próximos Passos Recomendados

5.1 Curto Prazo (Fase 1 - 2 semanas)

1. Finalizar Checkout

- [x] Página de checkout existente
- [] Validar todos os campos de endereço
- [] Adicionar validação de CEP japonês

2. Logística básica

- ☐ Campo para código de rastreamento no dashboard do vendedor
- ☐ Exibir código no dashboard do comprador
- ☐ Link para tracking do carrier

3. Notificações

- ☐ E-mail ao vendedor quando pagamento confirmado
- ☐ E-mail ao comprador quando envio registrado

5.2 Médio Prazo (Fase 2 - 1 mês)

1. Stripe Connect

- ☐ Implementar onboarding de vendedores
- ☐ Payouts automáticos para vendedores
- ☐ Portal de gerenciamento de pagamentos

2. Tracking melhorado

- ☐ Integração com API de tracking
- ☐ Status automático de entrega
- ☐ Notificações de atualização

5.3 Longo Prazo (Fase 3 - 2-3 meses)

1. Logística avançada

- ☐ Parceria com warehouse
- ☐ Sistema de combined shipping
- ☐ Multi-carrier support

2. Features adicionais

- ☐ Seguro de envio
 - ☐ Devoluções/reembolsos
 - ☐ Suporte ao cliente integrado
-

6. Perguntas para Definição

6.1 Logística

1. Qual modelo de logística preferem? (Direct Ship vs Warehouse)
2. Já conhecem carriers no Japão para integrar?
3. Tem orçamento/parceria específica para logística?

6.2 Payments

1. Precisam de payouts imediatos ou pode esperar período de holding?
2. Vendedores precisam de dashboard financeiro?
3. PIX é prioritário ou cartão é suficiente por agora?

6.3 Prioridades

1. O que deve ser entregue para o próximo milestone?
 2. Quais features são "must-have" vs "nice-to-have"?
-

7. Anexos

Arquivos Relevantes

- `src/pages/PaymentCheckout.tsx` - Página de checkout
- `src/components/PurchaseFlow.tsx` - Fluxo de compra
- `supabase/functions/stripe-checkout/index.ts` - API Stripe
- `supabase/functions/transactions/index.ts` - API Transações
- `src/lib/api.ts` - Cliente API frontend

Variáveis de Ambiente Necessárias

```
VITE_STRIPE_PUBLIC_KEY=pk_...  
STRIPE_SECRET_KEY=sk_...  
STRIPE_WEBHOOK_SECRET=whsec_...  
APP_URL=https://...
```

Nota: Este documento será atualizado conforme as decisões forem tomadas.